AI-AUGMENTED BACKEND DEVELOPMENT
AI BLOGNEST API
PHASE 5 – PROJECT DEVELOPMENT

ROLE	TEAM MEMBER
TEAM LEADER	Anisha.P
TEAM MEMBER	Bavani.C
TEAM MEMBER	Hemalatha.S
TEAM MEMBER	Jagasri.M

Submitted as a phase-wise project submission.

1. Introduction

The project Development phase implements the main server-side functionality of AI BlogNest API using Node.js and Express.js.

2. Server Development

- Create and configure the Express application.

```
app.js
src > app.js > ...
1   const express = require('express');
2   const cors = require('cors');
3   const morgan = require('morgan');
4   const authRoutes = require('./routes/authRoutes');
5   const blogRoutes = require('./routes/blogRoutes');
6   const aiRoutes = require('./routes/aiRoutes');
7   const errorMiddleware = require('./middleware/errorMiddleware');
8
9   const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22    res.json({ message: 'AI BlogNest API is running' });
23  });
24
25  module.exports = app;
26
```

- Configure middleware for JSON request handling and CORS.
- Start the server on the required port.
- Create organized routes and controllers.

```
server.js
src > server.js > ...
1   const dotenv = require('dotenv');
2   const app = require('./app');
3   const connectDB = require('./config/db');
4
5   dotenv.config();
6   connectDB();
7
8   const PORT = process.env.PORT || 5000;
9   app.listen(PORT, () => {
10    console.log(`Server running on port ${PORT}`);
11  });
12
```

3. API Development

- Implement user registration and login APIs.
- Implement blog creation, retrieval, update and deletion APIs.
- Implement category and tag management.
- Implement comment-related APIs.
- Return structured JSON responses.

4. Authentication & Authorization

- Use JWT for stateless authentication.
- Validate authenticated requests.
- Apply role-based access for Admin, Editor, Author and Reader.
- Restrict protected operations according to user roles.

5. Validation & Error Handling

- Validate incoming request data.
- Handle invalid requests and missing resources.
- Use consistent HTTP status codes and error messages.
- Apply input sanitization where required.

6. Expected Outcome

A functional REST backend is developed with organized routes, controllers, authentication, authorization and error handling.

7. Conclusion

This phase converts the planned architecture into working backend services that can communicate with the database and AI layer.